4. Vertex AI 소개 및 개요

```
!pip install datasets torchserve torch-model-archiver torch-workflow-archiver nvgpu
```

```
Collecting datasets
  Downloading datasets-3.2.0-py3-none-any.whl.metadata (20 kB)
Requirement already satisfied: torchserve in /usr/local/lib/python3.10/dist-packages (0.12.0)
Requirement already satisfied: torch-model-archiver in /usr/local/lib/python3.10/dist-packages (0.12.0)
Collecting torch-workflow-archiver
  Downloading torch_workflow_archiver-0.2.15-py3-none-any.whl.metadata (1.5 kB)
Collecting nvgpu
  Downloading nvgpu-0.10.0.tar.gz (8.4 kB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from datasets) (3.16.1)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from datasets) (1.26.4)
Collecting pyarrow>=15.0.0 (from datasets)
  Downloading pyarrow-18.1.0-cp310-cp310-manylinux_2_28_x86_64.whl.metadata (3.3 kB)
Collecting dill<0.3.9,>=0.3.0 (from datasets)
  Downloading dill-0.3.8-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from datasets) (2.1.4)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.10/dist-packages (from datasets) (2.32
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.10/dist-packages (from datasets) (4.66.5)
Collecting xxhash (from datasets)
  Downloading xxhash-3.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (12 kB)
Collecting multiprocess<0.70.17 (from datasets)
  Downloading multiprocess-0.70.16-py310-none-any.whl.metadata (7.2 kB)
Requirement already satisfied: fsspec<=2024.9.0,>=2023.1.0 in /usr/local/lib/python3.10/dist-packages (from fsspec
Requirement already satisfied: aiohttp in /usr/local/lib/python3.10/dist-packages (from datasets) (3.10.5)
Requirement already satisfied: huggingface-hub>=0.23.0 in /usr/local/lib/python3.10/dist-packages (from datasets
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from datasets) (24.1)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from datasets) (6.0.2)
Requirement already satisfied: Pillow in /usr/local/lib/python3.10/dist-packages (from torchserve) (10.4.0)
Requirement already satisfied: psutil in /usr/local/lib/python3.10/dist-packages (from torchserve) (5.9.5)
Requirement already satisfied: wheel in /usr/local/lib/python3.10/dist-packages (from torchserve) (0.44.0)
Requirement already satisfied: enum-compat in /usr/local/lib/python3.10/dist-packages (from torch-model-archive
Collecting ansi2html (from nvgpu)
  Downloading ansi2html-1.9.2-py3-none-any.whl.metadata (3.7 kB)
Collecting arrow (from nvgpu)
  Downloading arrow-1.3.0-py3-none-any.whl.metadata (7.5 kB)
Requirement already satisfied: flask in /usr/local/lib/python3.10/dist-packages (from nvgpu) (2.2.5)
Collecting flask_restful (from nvgpu)
  Downloading Flask_RESTful-0.3.10-py2.py3-none-any.whl.metadata (1.0 kB)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from nvgpu) (1.16.0)
Requirement already satisfied: termcolor in /usr/local/lib/python3.10/dist-packages (from nvgpu) (2.4.0)
Requirement already satisfied: tabulate in /usr/local/lib/python3.10/dist-packages (from nvgpu) (0.9.0)
Collecting pynvml (from nvgpu)
  Downloading pynvml-12.0.0-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp-
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datase
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datas
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.10/dist-packages (from aiohttp->da
Requirement already satisfied: yarl<2.0,>=1.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets
Requirement already satisfied: async-timeout<5.0,>=4.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp-
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from hugg
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from reques
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->d
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests>=2.
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests>=2.
Requirement already satisfied: python-dateutil>=2.7.0 in /usr/local/lib/python3.10/dist-packages (from arrow->nv
Collecting types-python-dateutil>=2.8.10 (from arrow->nvgpu)
  Downloading types_python_dateutil-2.9.0.20241206-py3-none-any.whl.metadata (2.1 kB)
```

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import seaborn as sns

from torch.utils.data import DataLoader
from transformers import BatchEncoding, BertTokenizer, BertForSequenceClassification, AdamW
from sklearn.metrics import confusion_matrix
from datasets import load_dataset
from tqdm import tqdm
from typing import TypedDict
import matplotlib.pyplot as plt
```

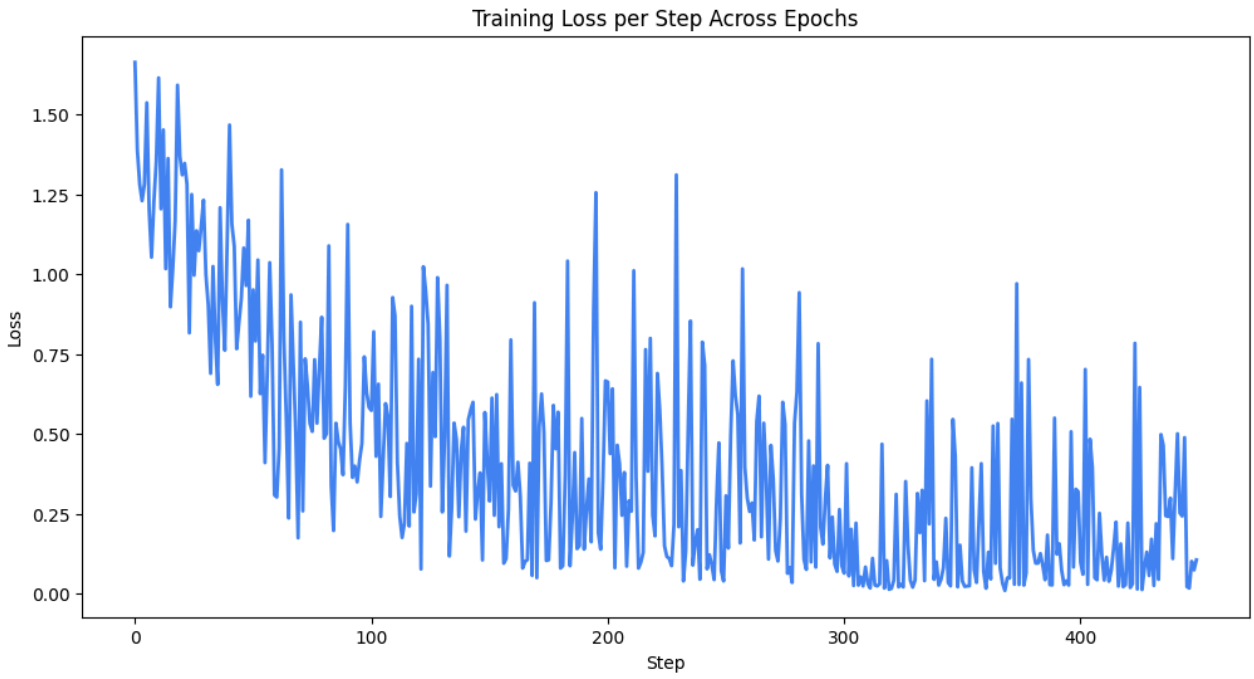
```
dataset = load_dataset("ag_news")
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=4)
optimizer = AdamW(model.parameters(), lr=5e-5)
criterion = torch.nn.CrossEntropyLoss()

class DatasetItem(TypedDict):
    text: str
    label: str


def preprocess_data(dataset_item: DatasetItem) -> dict[str, torch.Tensor]:
    return tokenizer(dataset_item["text"], truncation=True, padding="max_length", return_tensors="pt")


train_dataset = dataset["train"].select(range(1200)).map(preprocess_data, batched=True)
test_dataset = dataset["test"].select(range(800)).map(preprocess_data, batched=True)

train_dataset.set_format("torch", columns=["input_ids", "attention_mask", "label"])
test_dataset.set_format("torch", columns=["input_ids", "attention_mask", "label"])

train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=8, shuffle=False)
```

```
/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:89: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/t
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md: 100% 8.07k/8.07k [00:00<00:00, 641kB/s]
train-00000-of-00001.parquet: 100% 18.6M/18.6M [00:01<00:00, 18.2MB/s]
test-00000-of-00001.parquet: 100% 1.23M/1.23M [00:00<00:00, 17.7MB/s]
Generating train split: 100% 120000/120000 [00:00<00:00, 593546.98 examples/s]
Generating test split: 100% 7600/7600 [00:00<00:00, 243797.07 examples/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 3.04kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 1.29MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 1.37MB/s]
config.json: 100% 570/570 [00:00<00:00, 37.2kB/s]
/usr/local/lib/python3.10/dist-packages/transformers/tokenization_utils_base.py:1601: FutureWarning: `clean_up_to
  warnings.warn(
model.safetensors: 100% 440M/440M [00:05<00:00, 101MB/s]
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
/usr/local/lib/python3.10/dist-packages/transformers/optimization.py:591: FutureWarning: This implementation of A
  warnings.warn(
Map: 100% 1200/1200 [00:01<00:00, 801.50 examples/s]
Map: 100% 800/800 [00:00<00:00, 860.77 examples/s]
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

num_epochs = 3
losses: list[float] = []

for epoch in range(num_epochs):
    model.train()
    total_loss = 0
    for batch in tqdm(train_loader, desc=f"Epoch {epoch + 1}"):
        inputs = {key: batch[key].to(device) for key in batch}
        labels = inputs.pop("label")
        outputs = model(**inputs, labels=labels)
        loss = outputs.loss
        total_loss += loss.item()
        losses.append(loss.item())

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    average_loss = total_loss / len(train_loader)
    print(f"Epoch {epoch + 1}, Average Loss: {average_loss}")
```

```
Epoch 1: 100%|██████████| 150/150 [01:43<00:00,  1.45it/s]
Epoch 1, Average Loss: 0.7462436061600844
Epoch 2: 100%|██████████| 150/150 [01:42<00:00,  1.46it/s]
Epoch 2, Average Loss: 0.3459869069854418
Epoch 3: 100%|██████████| 150/150 [01:42<00:00,  1.46it/s]Epoch 3, Average Loss: 0.17397155596564212
```

```
plt.figure(figsize=(12, 6))
plt.plot(losses, color="#4285f4", linewidth=2)
plt.xlabel("Step")
plt.ylabel("Loss")
plt.title("Training Loss per Step Across Epochs")
plt.show()
```




```
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for batch in tqdm(test_loader, desc="Evaluating"):
        inputs = {key: batch[key].to(device) for key in batch}
        labels = inputs.pop("label")
        outputs = model(**inputs, labels=labels)
        logits = outputs.logits
        predicted_labels = torch.argmax(logits, dim=1)
        correct += (predicted_labels == labels).sum().item()
        total += labels.size(0)

accuracy = correct / total

print("")
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

```
Evaluating: 100%|██████████| 100/100 [00:21<00:00,  4.74it/s]
Test Accuracy: 79.62%
```

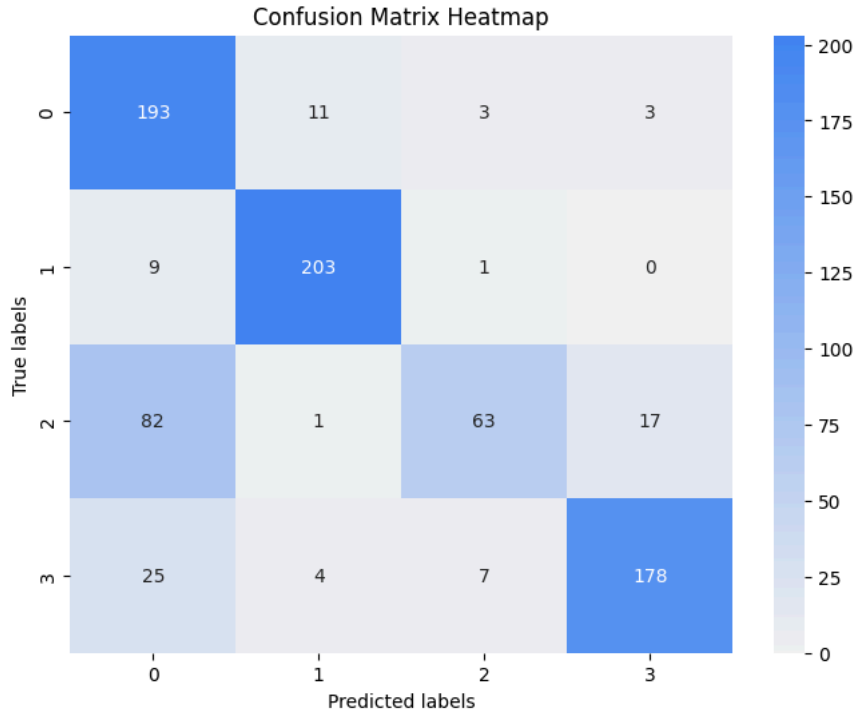
```
test_input = "[Official] 'Legendary Coach Resigns → Appoints New Commander' Suwon Completes Coaching Staff... Sc
test_input_processed = tokenizer(test_input, truncation=True, padding="max_length", return_tensors="pt").to(devi
logits = model(**test_input_processed).logits
print(logits)
predicted_labels = torch.argmax(logits, dim=1)

labeling_mapper = ["world", "sports", "business", "sci/tech"]
print(labeling_mapper[predicted_labels[0]])
```

```
tensor([[ 2.3975,  2.9687, -2.3953, -3.2521]], device='cuda:0',
       grad_fn=<AddmmBackward0>)
sports
```

```
all_predictions: list[int] = []
all_labels: list[int] = []

with torch.no_grad():
    for batch in tqdm(test_loader, desc="Evaluating"):
        inputs = {key: batch[key].to(device) for key in batch}
        labels = inputs.pop("label")
        outputs = model(**inputs)
        logits = outputs.logits
        predicted_labels = torch.argmax(logits, dim=1)

        all_predictions.extend(predicted_labels.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())
```

```
Evaluating: 100%|██████████| 100/100 [00:20<00:00,  4.80it/s]
```

```
conf_matrix = confusion_matrix(all_labels, all_predictions)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt="g", cmap=sns.light_palette("#4285f4", as_cmap=True))
plt.xlabel("Predicted labels")
plt.ylabel("True labels")
plt.title("Confusion Matrix Heatmap")
plt.show()
```



TorchServe Archive

```
# 모델 체크포인트 저장.
model_save_path = "model.pth"
torch.save(model.state_dict(), model_save_path)
```

```
%%writefile handler.py
import json
import logging

import torch
from ts.context import Context
from ts.torch_handler.base_handler import BaseHandler
from transformers import BatchEncoding, BertTokenizer, BertForSequenceClassification

logging.basicConfig(level=logging.INFO)

class ModelHandler(BaseHandler):
    def __init__(self):
        self.initialized = False
        self.tokenizer = None
        self.model = None

    def initialize(self, context: Context):
        properties = context.system_properties
```

```
            model_dir = properties.get("model_dir")
            self.initialized = True
            self.tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
            self.model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=4)
            model_path = model_dir + "/model.pth"
            self.model.load_state_dict(torch.load(model_path))
            self.model.to(torch.device("cuda" if torch.cuda.is_available() else "cpu"))
            self.model.eval()

        def preprocess(self, texts: list[str]) -> BatchEncoding:
            logging.info("preprocess", texts)

            inputs = self.tokenizer(texts, truncation=True, padding=True, max_length=512, return_tensors="pt")
            device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
            return inputs.to(device)

        def inference(self, input_batch: BatchEncoding) -> torch.Tensor:
            with torch.no_grad():
                outputs = self.model(**input_batch)
                logging.info("inference", outputs)
                return outputs.logits

        def postprocess(self, inference_output: torch.Tensor) -> list[dict[str, float]]:
            logging.info("postprocess", inference_output)

            probabilities = torch.nn.functional.softmax(inference_output, dim=1)
            return [{"label": int(torch.argmax(prob)), "probability": float(prob.max())} for prob in probabilities]
```

```
Overwriting handler.py
```

```
# bert vocab 파일 (아티팩트)

!wget https://raw.githubusercontent.com/microsoft/SDNet/master/bert_vocab_files/bert-base-uncased-vocab.txt \
    -O bert-base-uncased-vocab.txt
```

```
--2024-12-11 00:17:26--  https://raw.githubusercontent.com/microsoft/SDNet/master/bert_vocab_files/bert-base-unca
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 231508 (226K) [text/plain]
Saving to: 'bert-base-uncased-vocab.txt'

bert-base-uncased-v 100%[===================>] 226.08K   953KB/s    in 0.2s

2024-12-11 00:17:27 (953 KB/s) - 'bert-base-uncased-vocab.txt' saved [231508/231508]
```

```
# Torch serve archiver의 최종 파일 저장 위치.
# .mar 확장자 파일을 생성하여 패키징 하게 된다.

!mkdir -p model-store
```

Register to Vertex AI

```
!torch-model-archiver \
    --model-name model \
    --version 1.0 \
    --serialized-file model.pth \
    --handler ./handler.py \
    --extra-files "bert-base-uncased-vocab.txt" \
    --export-path model-store \
    -f
```

```
# from google.colab import auth
# auth.authenticate_user()
```

```
from google.cloud import storage, aiplatform
from google.oauth2 import service_account
```

```
credentials = service_account.Credentials.from_service_account_file("credentials.json")
```

```
PROJECT_ID = ""                    # @param {type: "s
LOCATION = ""                                       #
BUCKET_NAME = ""   # @param {type: "string"}
MODEL_FILE_NAME = ""               # @param {type: "st
```

PROJECT_ID

여기에 text 입력

LOCATION

여기에 text 입력

BUCKET_NAME

MODEL_FILE_NAME

여기에 text 입력 여기에 text 입력

코드 숨기기

```
storage_client = storage.Client(credentials=credentials)
bucket = storage_client.bucket(BUCKET_NAME)
```

```
def upload_blob(source_file_name: str, destination_blob_name: str) -> None:
    blob = bucket.blob(destination_blob_name)
    blob.upload_from_filename(source_file_name)
    print(f"File {source_file_name} uploaded to {destination_blob_name}.")
```

```
upload_blob("model-store/model.mar", f"models/{MODEL_FILE_NAME}")
```

```
File model-store/model.mar uploaded to models/model.mar.
```

```
aiplatform.init(
    project=PROJECT_ID,
    location=LOCATION,
    credentials=credentials,
)
```

```
model_path = f"gs://{BUCKET_NAME}/models"
registry_model = aiplatform.Model.upload(
    display_name="AG News Classification",
    artifact_uri=model_path,
    serving_container_image_uri="asia-northeast3-docker.pkg.dev/gde-project-aicloud/mlops-quicklab/trainer:1.0.3
    is_default_version=True,
    version_aliases=["v1"],
    version_description="A news category classification model",
    serving_container_predict_route="/predictions/model",
    serving_container_health_route="/ping",
)
```

```
INFO:google.cloud.aiplatform.models:Creating Model
INFO:google.cloud.aiplatform.models:Create Model backing LRO: projects/634821526652/locations/asia-northeast3/mod
INFO:google.cloud.aiplatform.models:Model created. Resource name: projects/634821526652/locations/asia-northeast3
INFO:google.cloud.aiplatform.models:To use this Model in another session:
INFO:google.cloud.aiplatform.models:model = aiplatform.Model('projects/634821526652/locations/asia-northeast3/mod
```

```
DEPLOY_COMPUTE = "n1-standard-2"
DEPLOY_ACCELERATOR = "NVIDIA_TESLA_T4"
```

```
endpoint = aiplatform.Endpoint.create(
    display_name="ag-news-category-classification",
    project=PROJECT_ID,
    location=LOCATION,
)
```

```
INFO:google.cloud.aiplatform.models:Creating Endpoint
INFO:google.cloud.aiplatform.models:Create Endpoint backing LRO: projects/634821526652/locations/asia-northeast3/
```

```
deployment = registry_model.deploy(
    endpoint=endpoint,
    machine_type=DEPLOY_COMPUTE,
    min_replica_count=1,
    max_replica_count=1,
    accelerator_type=DEPLOY_ACCELERATOR,
    accelerator_count=1,
    traffic_percentage=100,
    sync=True,
)
```

```
INFO:google.cloud.aiplatform.models:Deploying model to Endpoint : projects/634821526652/locations/asia-northeast3
INFO:google.cloud.aiplatform.models:Deploy Endpoint model backing LRO: projects/634821526652/locations/asia-north
INFO:google.cloud.aiplatform.models:Endpoint model deployed. Resource name: projects/634821526652/locations/asia-
```

```
endpoint.predict(instances=[
    "OpenAI releases AI video generator Sora to all customers"
])
```

```
Prediction(predictions=[{'label': 3.0, 'probability': 0.9921878576278687}],
deployed_model_id='7880257010875236352', metadata=None, model_version_id='1',
model_resource_name='projects/634821526652/locations/asia-northeast3/models/351848118934831104',
explanations=None)
```

Clean up

```
endpoint.undeploy_all()
endpoint.delete()
```

```
INFO:google.cloud.aiplatform.models:Undeploying Endpoint model: projects/634821526652/locations/asia-northeast3/e
INFO:google.cloud.aiplatform.models:Undeploy Endpoint model backing LRO: projects/634821526652/locations/asia-nor
INFO:google.cloud.aiplatform.models:Endpoint model undeployed. Resource name: projects/634821526652/locations/asi
INFO:google.cloud.aiplatform.base:Deleting Endpoint : projects/634821526652/locations/asia-northeast3/endpoints/9
INFO:google.cloud.aiplatform.base:Endpoint deleted. . Resource name: projects/634821526652/locations/asia-northea
INFO:google.cloud.aiplatform.base:Deleting Endpoint resource: projects/634821526652/locations/asia-northeast3/end
INFO:google.cloud.aiplatform.base:Delete Endpoint backing LRO: projects/634821526652/locations/asia-northeast3/op
INFO:google.cloud.aiplatform.base:Endpoint resource projects/634821526652/locations/asia-northeast3/endpoints/968
```

```
registry_model.delete()
```

```
INFO:google.cloud.aiplatform.base:Deleting Model : projects/634821526652/locations/asia-northeast3/models/3518481
INFO:google.cloud.aiplatform.base:Model deleted. . Resource name: projects/634821526652/locations/asia-northeast3
INFO:google.cloud.aiplatform.base:Deleting Model resource: projects/634821526652/locations/asia-northeast3/models
INFO:google.cloud.aiplatform.base:Delete Model backing LRO: projects/634821526652/locations/asia-northeast3/model
INFO:google.cloud.aiplatform.base:Model resource projects/634821526652/locations/asia-northeast3/models/351848118
```